

JOMO KENYATTA UNIVERSITY OF AGRICULTURE & TECHNOLOGY

COMPUTING DEPARTMENT

ICS 2313: SOFTWARE TESTING & QUALITY ASSURANCE

COHORT SPACE SYSTEM DESIGN DOCUMENT

Project Name	Cohort Space
Prepared By	Kelvin Otieno Abidha; Cynthia Vaati Mbithe; Munene Josphat; Dekow Mohamed
Purpose	Detailed system design for the proposed academic project management hub
Approved Technology Stack	Frontend: React.js Backend: Node.js Database: PostgreSQL
Supporting Tools	Figma, GitHub, Selenium, Google Chrome
Version	1.0

This design document extends the project proposal and translates the approved idea into a structure that can be implemented, tested, and presented consistently.

Table of Contents

1. Introduction
2. Design goals and scope
3. Approved technology stack
4. System architecture
5. Data and relationships
6. Process flow analysis
7. Forms, reports, and interface design
8. Validation, security, and testing notes
9. Conclusion

1. Introduction

Cohort Space is designed as a focused academic workspace for student project groups. Instead of letting project discussions, files, and deadlines disappear inside ordinary chat applications, the platform brings them together in one controlled environment. The main idea is simple: every team should be able to see what needs to be done, who is responsible for it, what resources are available, and how close the group is to its next milestone.

This document presents the logical design of the system. It covers the chosen stack, the major data entities, the relationships between those entities, the movement of information through the platform, and the main user-facing forms and reports. The writing is intentionally practical so that it can support both implementation work and classroom presentation.

2. Design goals and scope

- Keep team work organized within a single workspace protected by a team PIN.
- Reduce social loafing by making task ownership and completion visible.
- Support milestone-based planning so that project deadlines are not separated from daily task execution.
- Provide a simple interface that students can understand quickly without training.
- Retain the approved scope from the proposal and avoid drifting into features that were not originally justified.

3. Approved technology stack

The design keeps the same stack that appears in the proposal so that the document remains consistent with the earlier submission.

Layer / Tool	Selected Technology	Reason for Use
Frontend	React.js	Builds a responsive and component-based user interface for dashboards, forms, and progress views.
Backend	Node.js	Handles application logic, request processing, authentication checks, and data exchange between the interface and the database.
Database	PostgreSQL	Stores structured data such as workspaces, members, tasks, milestones, and uploaded resource records.
UI/UX Design	Figma	Supports wireframing, layout planning, and collaborative interface design before coding begins.
Version Control	GitHub	Tracks changes, supports collaboration, and keeps the project codebase organized.
Testing	Selenium	Automates browser-based testing for important flows such as login, task updates, and report viewing.
Browser for Development and QA	Google Chrome	Used during development and debugging because of its mature

Layer / Tool	Selected Technology	Reason for Use
		inspection tools.

Design note:

The document does not introduce a different backend, database, or frontend stack because the proposal already established React.js, Node.js, and PostgreSQL as the official implementation base.

4. System architecture

At a high level, the platform follows a straightforward three-layer structure. The user interacts with React.js screens in the browser. Those screens send requests to the Node.js application, which applies business rules and talks to PostgreSQL for storage and retrieval. Files and shared links are represented in the database as resource records so that they stay tied to the correct workspace and uploader.

The design favors clarity over complexity. Since the system is meant for academic teams, the architecture should be stable, easy to demonstrate, and simple enough to test without a heavy deployment footprint.

Component	Main Responsibility	Input	Output
React.js client	Displays pages, forms, task board, milestone views, and reports	User clicks, typed values, status changes	Requests sent to backend and updated screens
Node.js server	Validates requests, enforces rules, coordinates data operations	HTTP requests from the client	JSON responses and processed records
PostgreSQL database	Stores persistent project data	Insert, update, delete, and select queries	Structured project data for the application

5. Data and relationships

The database design centers around the workspace. A workspace represents one team project area. Every member, task, milestone, resource, and activity log entry belongs to a workspace. This makes the data model easy to reason about and keeps isolation between project groups clear.

Cohort Space: Core Data Relationships

Design document figure

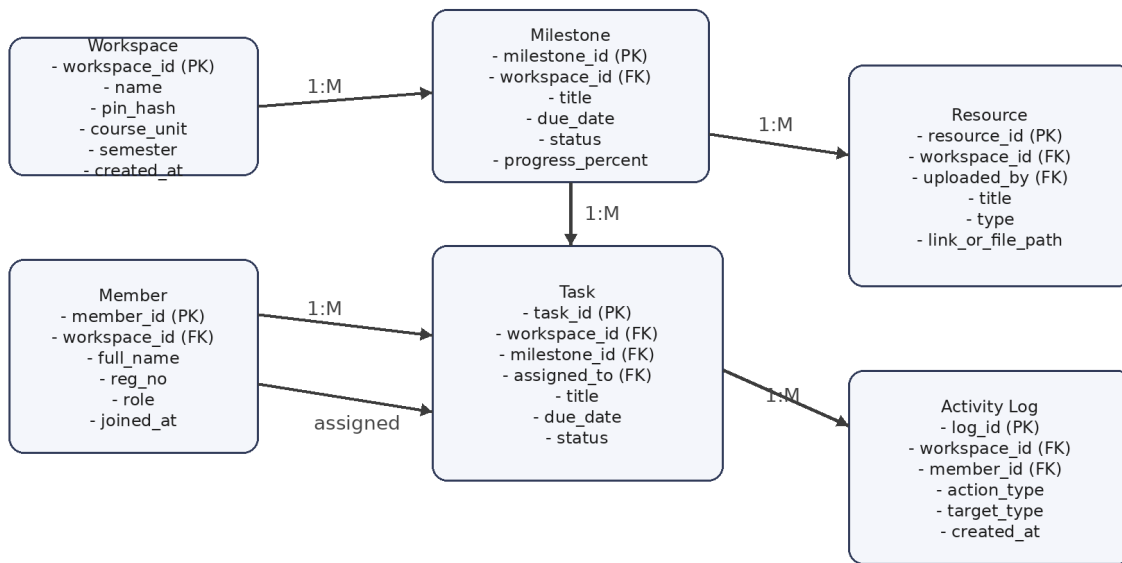


Figure 1: Core entity relationship overview.

5.1 Entity descriptions

Entity	Purpose	Important Fields
Workspace	Stores the identity of a project team area and controls access to it.	workspace_id, name, pin_hash, course_unit, semester, created_at
Member	Stores information about each student inside a workspace.	member_id, workspace_id, full_name, reg_no, role, joined_at
Milestone	Represents a major deadline or checkpoint for the team.	milestone_id, workspace_id, title, due_date, status, progress_percent
Task	Represents assignable work items linked to a member and optionally a milestone.	task_id, workspace_id, milestone_id, assigned_to, title, due_date, status
Resource	Represents a file upload or saved link relevant to the project.	resource_id, workspace_id, uploaded_by, title, type, link_or_file_path
Activity Log	Captures important actions for traceability and accountability.	log_id, workspace_id, member_id, action_type, target_type, created_at

5.2 Relationship summary

- One workspace can have many members.
- One workspace can have many milestones.

- One workspace can have many tasks.
- One milestone can contain many tasks.
- One member can be assigned many tasks, but each task is assigned to one member at a time.
- One workspace can have many resources and many activity log entries.

5.3 Table design

Workspace

Field	Data Type	Constraint	Description
workspace_id	INTEGER / SERIAL	PK	Unique identifier for the workspace
name	VARCHAR(100)	Required	Project or team name
pin_hash	VARCHAR(255)	Required	Securely stored hashed PIN
course_unit	VARCHAR(100)	Optional	Course unit linked to the project
semester	VARCHAR(40)	Optional	Academic period
created_at	TIMESTAMP	Required	Date and time of creation

Member

Field	Data Type	Constraint	Description
member_id	INTEGER / SERIAL	PK	Unique identifier for a member
workspace_id	INTEGER	FK	Links member to one workspace
full_name	VARCHAR(120)	Required	Student full name
reg_no	VARCHAR(30)	Required	Registration number
role	VARCHAR(30)	Required	Role such as leader or member
joined_at	TIMESTAMP	Required	When the member joined the workspace

Milestone

Field	Data Type	Constraint	Description
milestone_id	INTEGER / SERIAL	PK	Unique identifier for a milestone
workspace_id	INTEGER	FK	Owning workspace
title	VARCHAR(120)	Required	Milestone title
description	TEXT	Optional	Extra milestone notes
due_date	DATE	Required	Target completion date

Field	Data Type	Constraint	Description
status	VARCHAR(20)	Required	Planned, Active, Completed, Overdue
progress_percent	INTEGER	Required	Overall progress from 0 to 100

Task

Field	Data Type	Constraint	Description
task_id	INTEGER / SERIAL	PK	Unique identifier for a task
workspace_id	INTEGER	FK	Owning workspace
milestone_id	INTEGER	FK	Related milestone
assigned_to	INTEGER	FK	Member responsible for the task
title	VARCHAR(120)	Required	Task title
description	TEXT	Optional	Task details
due_date	DATE	Required	Task deadline
status	VARCHAR(20)	Required	To Do, In Progress, Done
priority	VARCHAR(20)	Optional	Low, Medium, High

Resource

Field	Data Type	Constraint	Description
resource_id	INTEGER / SERIAL	PK	Unique identifier for a resource
workspace_id	INTEGER	FK	Owning workspace
uploaded_by	INTEGER	FK	Member who added the resource
title	VARCHAR(120)	Required	Resource title
type	VARCHAR(30)	Required	Link, PDF, DOCX, Image, Other
link_or_file_path	TEXT	Required	Saved URL or file location
uploaded_at	TIMESTAMP	Required	Upload date and time

Activity Log

Field	Data Type	Constraint	Description
log_id	INTEGER / SERIAL	PK	Unique identifier for the log item

Field	Data Type	Constraint	Description
workspace_id	INTEGER	FK	Owning workspace
member_id	INTEGER	FK	Actor who performed the action
action_type	VARCHAR(50)	Required	Created task, updated status, uploaded file, and related actions
target_type	VARCHAR(50)	Required	Task, milestone, resource, or workspace
target_id	INTEGER	Optional	Specific affected record
created_at	TIMESTAMP	Required	Time of action

6. Process flow analysis

The main processes in Cohort Space are designed to follow how students actually work on projects. A team first gets into its workspace, then creates milestones and tasks, adds supporting files or links, monitors progress, and finally reviews status through reports and contribution views.

Cohort Space: Main User Flow

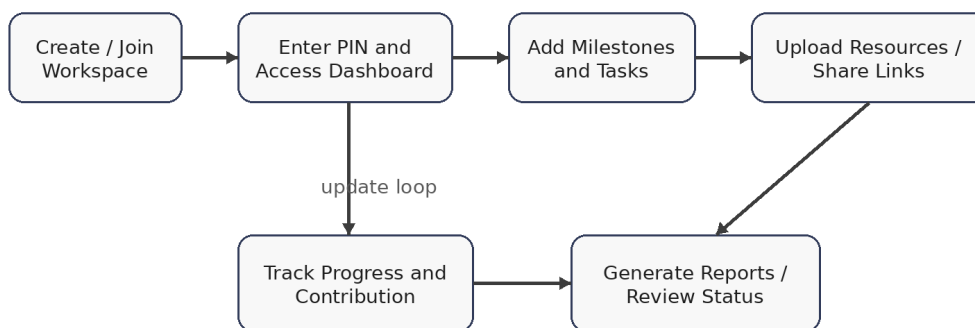


Figure 2: Main system flow from access to reporting.

6.1 Workspace access flow

Step	Description	Expected Result
1	User opens the application and chooses to create or join a workspace.	The system displays the access form.
2	User enters workspace details or the team PIN.	The backend validates the request.
3	If the credentials are correct, the system loads the dashboard.	Authorized user gains access to project content.
4	If validation fails, the system shows a clear error message.	Unauthorized access is blocked without ambiguity.

6.2 Task management flow

Step	Description	Expected Result
1	A member creates a task and links it to a milestone when necessary.	The task record is stored in the database.
2	The task is assigned to a member and given a due date.	Ownership becomes visible to the team.
3	The assigned member updates status as work continues.	Progress moves across To Do, In Progress, and Done states.
4	The dashboard recalculates completion indicators.	The team sees current progress at a glance.

6.3 Resource vault flow

Step	Description	Expected Result
1	A member uploads a file or saves a useful link.	A new resource record is created.
2	The system tags the resource with title, type, workspace, and uploader.	The item is traceable and easy to retrieve.
3	Other members browse or open the resource from the vault.	Important materials do not get lost in chats.

6.4 Reporting flow

Step	Description	Expected Result
1	The dashboard gathers task, milestone, and member activity data.	Current project state is summarized.
2	The reporting module calculates completion rates and overdue items.	Managers and members get actionable insight.
3	The system displays contribution	The group can spot imbalances early.

Step	Description	Expected Result
	figures per member.	

7. Forms, reports, and interface design

The interface should remain simple and familiar. Since the users are students managing coursework, the design should focus on fast understanding rather than decorative complexity. The screens below are the main ones needed for a complete first version of the system.

Cohort Space: Interface Outline

The dashboard layout is titled "Cohort Space Dashboard". It features a left sidebar with navigation buttons: Dashboard, Tasks, Milestones, Resources, Reports, and Settings. The main content area is divided into several sections:

- Summary Cards:** Four cards at the top showing "Open Tasks" (14), "Milestones Due" (3), "Completion Rate" (68%), and "Active Members" (6).
- Task Board:** A central section with three columns: "To Do", "In Progress", and "Done". Each column contains three "Task item" boxes.
- Upcoming Milestones:** A list on the right showing "Proposal Review - 12 Mar", "UI Mockup - 18 Mar", and "Testing Demo - 30 Mar".
- Recent Activity:** A list on the right showing "Kelvin uploaded chapter draft", "Cynthia completed ERD task", and "Josphat updated milestone status".

Figure 3: Suggested dashboard layout in the same simple style direction as the proposal.

Form	Purpose	Key Fields
Workspace creation / access form	Allows a team to create a workspace or enter an existing one.	Workspace name, team PIN, optional course unit, semester
Member form	Registers or lists members within the workspace.	Full name, registration number, role
Task form	Captures assignable work items.	Title, description, milestone, assigned member, due date, status, priority
Milestone form	Creates major deadlines and checkpoints.	Title, description, due date, status
Resource form	Stores a file or a reference link.	Title, type, file or URL, notes

Report / View	What It Shows	Why It Matters
Dashboard summary	Open tasks, active milestones, completion percentage, active members	Gives an immediate project health snapshot
Task board	Tasks grouped by status	Makes work movement visible and easy to follow
Milestone tracker	Upcoming deadlines and completion progress	Keeps the team aligned with semester expectations
Contribution report	Tasks completed per member and recent activity	Reduces social loafing and supports accountability
Resource vault list	All saved files and links with uploader details	Keeps shared materials organized and retrievable

7.3 Interface behaviour notes

- Every major page should carry the workspace name so members always know which project they are inside.
- Task and milestone statuses should use clear text labels; color can support meaning, but the wording should remain understandable on its own.
- Error messages should be short and direct, especially during workspace access and task updates.
- The layout should work cleanly on both laptops and mobile devices, in line with the responsiveness requirement from the proposal.
- Navigation should remain stable: Dashboard, Tasks, Milestones, Resources, Reports, and Settings are enough for the first release.

8. Validation, security, and testing notes

Because the platform stores team activity and shared project materials, validation and security cannot be left vague. Even though the access model is intentionally lightweight, the system still needs basic protection and predictable behaviour.

Area	Design Decision
PIN handling	Store the workspace PIN as a hash rather than plain text.
Authorization	Only members inside a workspace should view or modify its tasks, resources, and milestones.
Input validation	Required fields such as task title, due date, and assigned member should be validated before saving.
Auditability	Important actions should be captured in the activity log.
Testing approach	Selenium should automate access flow, task creation, task status changes, and report visibility checks.
Reliability	Database-backed storage ensures that tasks and resources remain available after logout or restart.

9. Conclusion

This design document has helped us clearly define how the Cohort Space Management System will work before the actual development begins. It has shown the main data involved, how different parts of the system relate to each other, the processes the system will perform, and the forms, reports, and interfaces that users will interact with. This makes it easier to move to the implementation stage with a clear understanding of what needs to be built.

As students, working on this document has also helped us gain practical experience in system analysis and design. It has allowed us to apply the concepts learned in class to a realistic project and understand the importance of proper planning before coding starts. Overall, this document acts as a guide for the next stages of the project and gives us confidence that the final system can be developed in a structured and effective way.